



Security Audit

YOM 3rd (Token)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Conclusion	23
Our Methodology	24
Disclaimers	26
About Hashlock	27

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The YOM team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

YOM is revolutionizing cloud gaming by pioneering the first decentralized cloud gaming infrastructure (DePIN). With YOM, games, immersive experiences, and virtual worlds can be streamed to any device, eliminating the need for expensive hardware like consoles (Xbox, PlayStation), powerful PCs, or dedicated apps (e.g., Steam, Apple Store).

Project Name: YOM

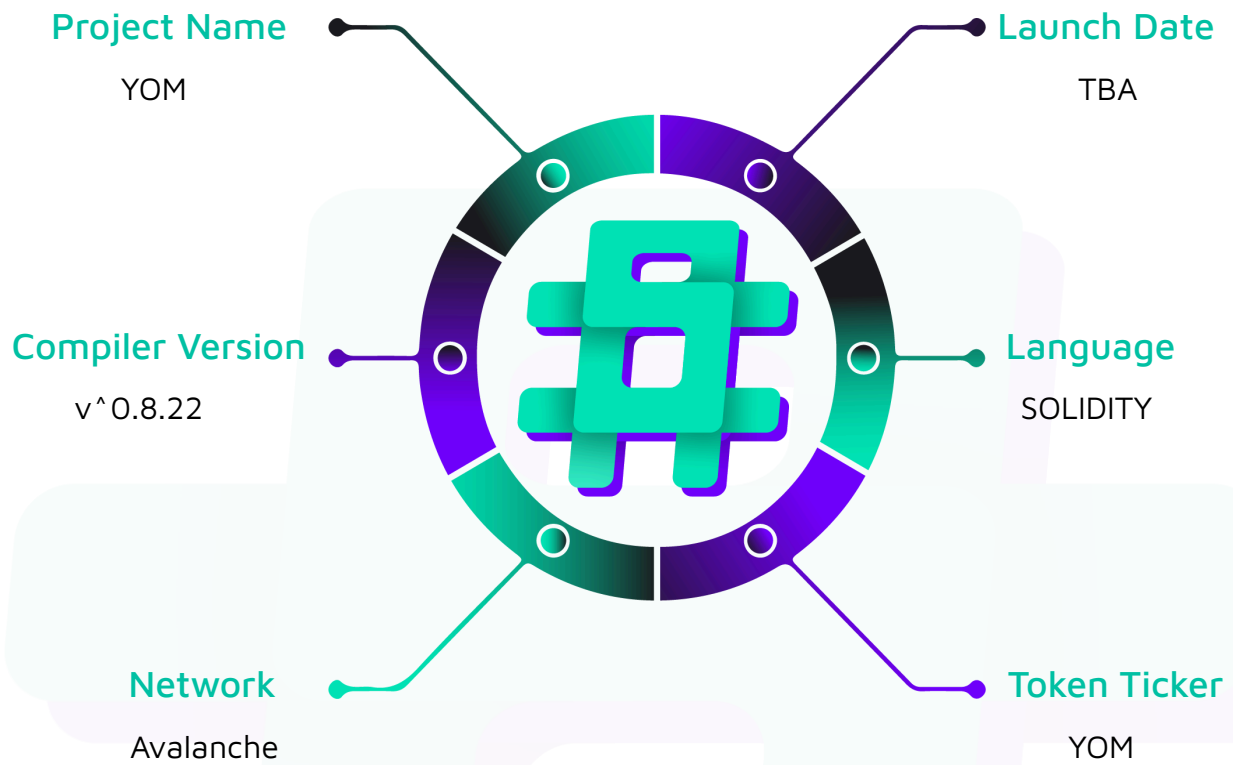
Project Type: Token

Compiler Version: ^0.8.22

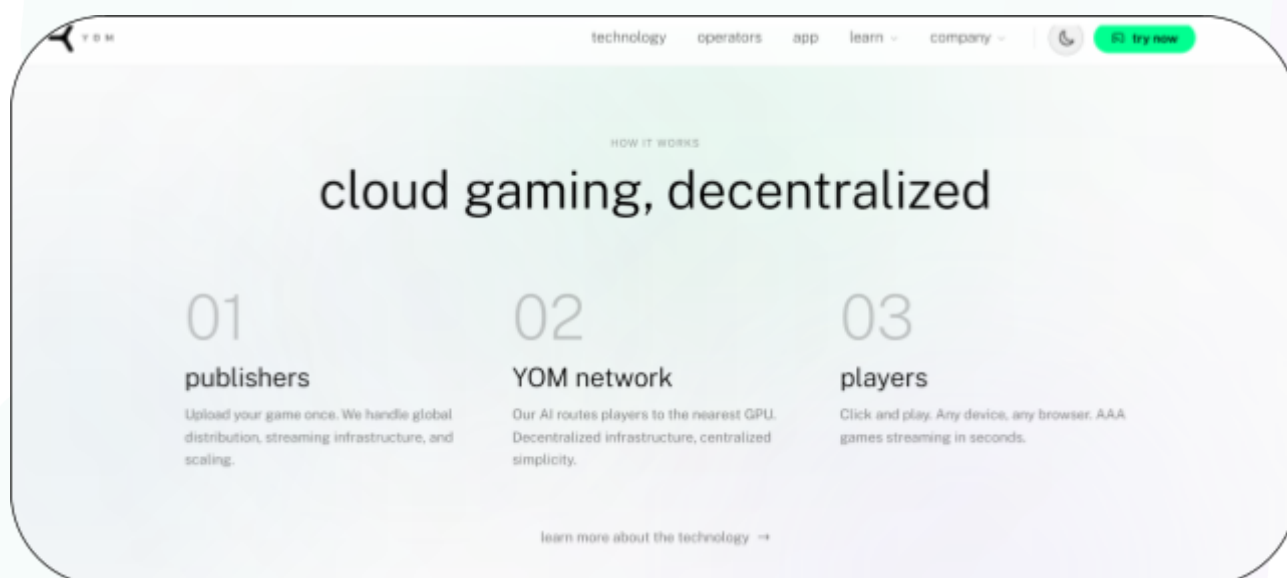
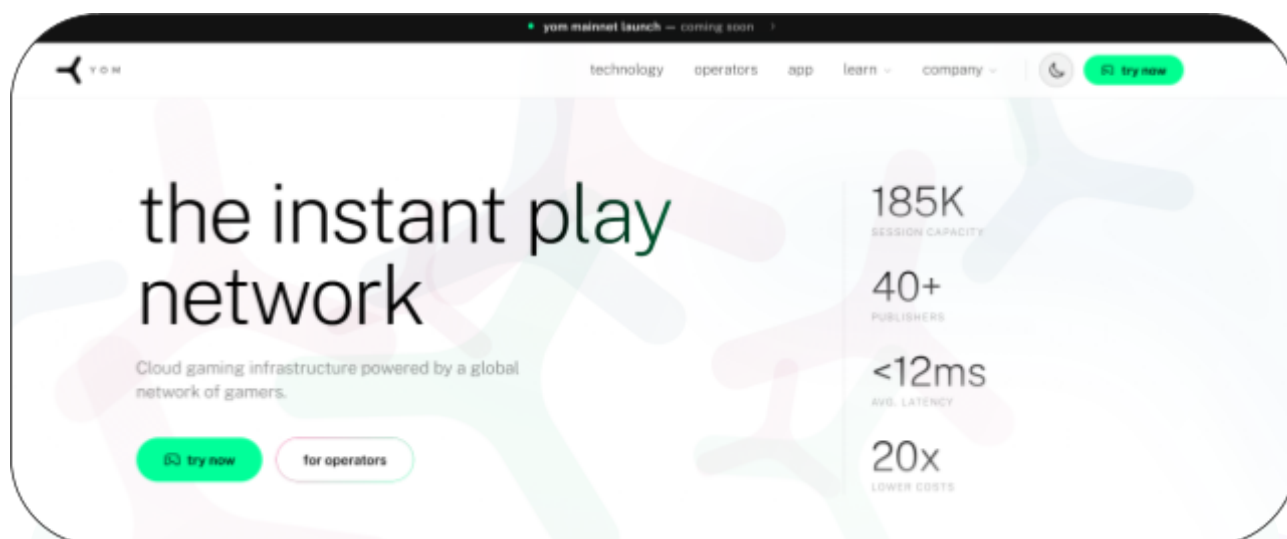
Website: www.yom.net

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the YOM project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	YOM Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	April, 2026
Contract 1	YOM.sol
Contract 2	YOMOFTAdapter.sol
Contract 3	YOMSecondaryOFT.sol
Audited GitHub Commit Hash	b1c3f244d876bdf73ff71d2c3e464a81dd519786
Fix Review GitHub Commit Hash	6e275b0d243da57c588b6036c97be274f1f0225b

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

3 Medium severity vulnerabilities

2 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Yom.sol Allows the Admin to: - Upgrade the smart contract - Set treasury and network addresses - Configure or disable launch protection (anti-sniper) - Freeze or blacklist specific accounts - Configure MEV protection (transfer cooldowns and block limits) - Exempt specific accounts from MEV rules - Rescue stuck ERC20 tokens or ETH Allows the Pauser to: - Pause and unpause all token transfers Allows the Payout system to: - Distribute rewards (splits YOM between the node operator, the network, and a burn address) Allows users to: - Transfer tokens (subject to pause, freeze, blacklist, and MEV rules) - Burn their own tokens	Contract achieves this functionality.

YOMOFTAdapter.sol Allows the Owner/Delegate to: - Manage LayerZero bridge configurations and settings (via inherited <code>'Ownable'</code> and <code>'OFTAdapter'</code> logic) Allows users to: - Bridge YOM tokens to other supported blockchains (locks their tokens in this adapter on the primary chain) - Receive YOM tokens bridged back from other chains (unlocks tokens from this adapter)	Contract achieves this functionality.
YOMSecondaryOFT.sol Allows the Owner/Delegate to: - Manage LayerZero bridge configurations and settings (via inherited <code>Ownable</code> and <code>OFT</code> logic) Allows the Bridge (LayerZero System) to: - Mint new YOM tokens on this secondary chain when a user bridges them over from the primary chain Allows users to: - Transfer YOM tokens normally on this secondary chain (acts as a standard ERC20 token) - Bridge YOM tokens back to the primary chain (which burns the tokens on this secondary chain)	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the YOM project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the YOM project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] YOM#_update - Zero-amount `transferFrom` enables MEV-cooldown censorship of any holder

Description

Because OpenZeppelin ERC20 allows `transferFrom(from, to, 0)` even when the caller has zero allowance, any third party can trigger your MEV protection bookkeeping for an arbitrary non-exempt holder without moving any tokens, and thereby make that holder's real transfer revert when included in the same block.

Vulnerability Details

In OpenZeppelin ERC20Upgradeable, `transferFrom(from, to, value)` always calls `_spendAllowance(from, spender, value)` and then `_transfer(from, to, value)`. When `value == 0`, `_spendAllowance` does not revert even if allowance is 0 because the check is `currentAllowance < value`, which is false for `0 < 0`. This means an attacker can call `transferFrom(victim, victim, 0)` with no approvals. Your token overrides `_update` and applies MEV protection even for amount `== 0`, and if neither side is MEV-exempt it updates `lastTransferBlock[from] = block.number` (and also increments `transfersPerBlock[from][block.number]`). If the attacker's zero-amount transaction is mined before the victim's real transfer in the same block, the victim's real transfer will revert due to `blocksSinceLastTransfer == 0` being less than the default `transferCooldown == 1`.

```
// OZ ERC20Upgradeable (v5.3.0)

function transferFrom(address from, address to, uint256 value) public virtual returns
(bool) {

    address spender = _msgSender();

    _spendAllowance(from, spender, value); // value==0 passes even if allowance==0
```

```
    _transfer(from, to, value);  
  
    return true;  
}
```

Impact

An unprivileged attacker can grief/censor a specific non-exempt holder's transfers by front-running each public-mempool transfer with `transferFrom(victim, victim, 0)`, causing the victim's real transfer to revert under the MEV cooldown logic when both land in the same block; this does not steal funds but can disrupt transfers and integrations that rely on timely token movements.

Recommendation

Ensure zero-amount transfers cannot mutate MEV protection state (and/or cannot be used as a third-party `transferFrom` against a victim), so that MEV throttling only applies to transfers that actually move tokens and are authorized by the sender.

Status

Resolved

[M-02] YOM#payout - `payout()` treasury unset fallback can DoS payouts under default MEV cooldown

Description

If `treasuryAddress` is not set, `payout()` silently uses `msg.sender` as the treasury and performs three internal transfers from that address; with the default `transferCooldown` = 1, the second transfer reverts unless the payout wallet is manually MEV-exempt (or cooldown is disabled), making this path broken by default and easy to misconfigure.

Vulnerability Details

In `payout()`, the contract selects `effectiveTreasury` as `treasuryAddress` when configured, otherwise it falls back to `msg.sender`. It then calls `_update(effectiveTreasury, ...)` three times in the same transaction (node share, network share, burn). Your `_update()` enforces MEV cooldown per sender and sets `lastTransferBlock[from] = block.number` after the first leg, so the second leg sees `blocksSinceLastTransfer == 0` and reverts when `transferCooldown` is 1 and neither side is MEV-exempt.

```
address effectiveTreasury = treasuryAddress != address(0) ? treasuryAddress : msg.sender;
_update(effectiveTreasury, nodeOperator, nodeShare);
_update(effectiveTreasury, networkAddress, networkShare); // can revert due to cooldown
_update(effectiveTreasury, address(0), burnedAmount);
```

Impact

If operations forget to set `treasuryAddress` (or rely on the fallback), payouts can unexpectedly spend the `PAYOUT_ROLE` wallet's own balance and may fail entirely due to the cooldown check on the second payout leg, resulting in a payout DoS until extra configuration is performed.

Recommendation

Avoid using `msg.sender` as a silent treasury fallback for payouts, and ensure the payout flow cannot trip MEV cooldown during its own multi-leg internal transfers (for example

by requiring a configured treasury that is MEV-exempt, or by excluding the payout flow from cooldown bookkeeping).

Status

Resolved



[M-03] YOM#_update - MEV protection becomes unusable or bypassed for DEX trading due to exemption logic in `\_update()`

Description

The MEV throttling is applied only when both from and to are non-exempt, which forces an operational choice for AMM trading: if the DEX pair is not exempt, the pair gets throttled as a sender and trading throughput breaks; if the pair is exempt to allow trading, then MEV throttling is skipped for all swaps involving the pair, so sandwich-style same-block buy and sell flows are not restricted by this mechanism.

Vulnerability Details

Every ERC20 movement goes through `\_update(from, to, amount)` via `transfer` and `transferFrom`. The MEV protection block is guarded by `if (!isMevExempt[from] && !isMevExempt[to])`, so it runs only when neither side is exempt. On a DEX buy, the pool/pair contract is typically the `from` address, so if the pool is not exempt the first swap in a block sets `lastTransferBlock[pool] = block.number` and a second swap in the same block reverts because `blocksSinceLastTransfer == 0` while the default `transferCooldown == 1`. To restore trading, the admin must exempt the pool, but then the MEV block is skipped for both directions of swaps because either `from` or `to` will be the exempt pool, meaning bot buy/sell activity involving the pool does not trigger cooldown or per-block transfer counting at all.

```
// YOM.sol

if (!isMevExempt[from] && !isMevExempt[to]) {

    // cooldown + per-block limits applied only when BOTH sides are non-exempt

    lastTransferBlock[from] = block.number;

}
```

Impact

In a real deployment with active AMM trading, leaving the pool non-exempt can severely degrade or break tradability by allowing only one pool-originating transfer per block under the default cooldown, while exempting the pool to allow normal trading causes

the MEV throttling to be skipped for all swaps involving that pool, making this “MEV protection” ineffective against same-block sandwich-style buy and sell flows around user swaps.

Recommendation

Do not rely on a single `isMevExempt` “either-side bypass” rule for both operational/system addresses and DEX trading; instead enforce MEV throttling based on the sender (or add a separate, explicit exemption category for AMM/router addresses) so that enabling DEX trading does not automatically disable throttling for all trades.

Status

Resolved

Low

[L-01] YOM#reinitializeV2 - `reinitializeV2()` is publicly callable (reinitializer slot can be consumed by anyone)

Vulnerability Details

`reinitializeV2()` is declared as `public reinitializer(2)` with no role restriction, so any external account can call it once and permanently consume the version-2 reinitializer slot; the direct state change is only `isFrozen[address(0)] = false`, but this can still cause an upgrade/migration race where a third party calls it before the team's planned upgrade transaction (especially if the team uses an atomic upgrade+call flow), forcing the team's reinitializer call to revert and breaking automated deployment/upgrade scripts.

Recommendation

Restrict `reinitializeV2()` to an admin role and emit an event for the migration state change so that only the intended operator can perform migrations and off-chain monitors can observe that the reinitializer has been executed.

Status

Resolved

[L-02] YOM# - Floating pragma (^0.8.22) in 'YOM.sol'

Vulnerability Details

'contracts/YOM.sol' uses 'pragma solidity ^0.8.22;', which allows compilation with any Solidity version from 0.8.22 up to (but not including) 0.9.0; this can lead to inconsistent builds if different compiler patch versions are used across environments.

Recommendation

Pin the pragma to a specific compiler version (for example 'pragma solidity 0.8.22;') and keep the build tooling aligned to the same version.

Status

Resolved

Centralisation

The YOM project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the YOM project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.